

0.107.4. La fonction couleurLutin() — une couleur des personnages

`#include <couleurLutin>` : `couleurLutin(palette, teinte, rouge, vert, bleu)` choisit une couleur des palettes des lutins (Game Boy Color).

1. Ce qu'il faut comprendre

Ce qu'elle fait : les lutins ont leurs **propres 8 palettes**. `couleurLutin(palette, teinte, rouge, vert, bleu)` en choisit une couleur. La **teinte 0 est transparente** : on voit le fond à travers.

La ligne à écrire : `#include <couleurLutin>`.

Ses arguments : la palette (0 à 7), la teinte (1 à 3), rouge, vert, bleu de 0 à 31.

Ce qu'elle coûte : environ **130 octets** avec la mise en place des lutins.

Essaie : fais le A vert : `couleurLutin(0, 3, 0, 31, 0);`.

On retrouve couleurLutin dans 3 leçons

[0.108](#), [104](#), [106](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **251 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l'allumage — la mise en route de la console
main()	45 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	31 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	16 o	← CopierTuiles — 1 lettre sur 43
DonneesTransfert	10 o	← InstallerTransfert — la petite routine qui copie les lutins

Lettres gravées : A

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes

Fonctions du langage appelées :

sprite()	× 1	23 o sur place, dans main()
couleurLutin()	× 1	12 o sur place, dans main()
image()	× 1	6 o sur place, dans main() + AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

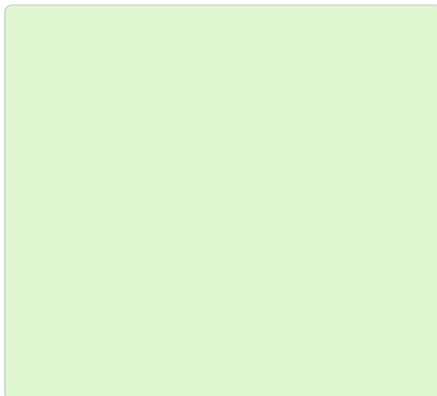
```
// ---- #include <couleurLutin> : une couleur des personnages ----  
// Un lutin est dans la palette 0 des lutins : sa teinte 3 devient bleue.  
  
#include <couleurLutin>    // couleurLutin() : une couleur des lutins  
#include <sprite>  
#include <ALPHABET>
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    couleurLutin(0, 3, 0, 0, 31);    // palette 0 des lutins, teinte 3 : bleu  
    sprite(0, 76, 68, ALPHABET[0]); // le A, dans cette palette  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un A bleu au milieu de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la cartouche tourne en couleur
- ✓ la teinte 3 de la palette 0 des lutins est bleue (31744)
- ✓ le lutin est posé

Ensuite : **0.107.5. La fonction `teindreLutin()` — un lutin dans une palette** — chapitre 0 : Avant tout. · [gameboy3](#) — écrire une cartouche Game Boy en C++